

AEGIS

Android Enterprise Guard & Intelligence Security
PROJECT BUILD PROMPT SEQUENCE — v2 Simplified Architecture

Document Type	Build & Learn Guide — Simplified Architecture
Phases	5 Build Phases (Dynamic Analysis removed)
Total Prompts	15 Prompts + 15 Checkpoints
Stack	Kotlin · Python · FastAPI · Kafka · React · Claude API
Key Change	All device data moves as JSON → Server first
Analysis	Static Engine + AI CLI run in parallel (same phase)
Logs	Streaming — important logs only, AI agent processes them
Dynamic	Removed — can be added later if needed

HOW TO USE THIS DOCUMENT

Each section contains (1) a BUILD PROMPT to give Claude to generate code, followed by (2) an UNDERSTANDING CHECKPOINT to help you fully learn what was built. Phases are ordered — complete each phase before starting the next. The two parallel tracks in Phase 3 should be started at the same time.

PH 01	Foundation — Kotlin Android Agent □ Weeks 1–4
-------	---

Build the core EDR agent running on every enrolled device. The agent collects device security data, app intelligence, and filters logs — then ships everything as JSON to the server.

Prompt 1.1 — Project Scaffold & Gradle Setup

□ COPY & PASTE TO CLAUDE You are a senior Android engineer. Scaffold a production-grade Android library module named "aegis-agent" using Kotlin and Gradle (Kotlin DSL). Requirements: - minSdk 26, targetSdk 34 - Hilt for dependency injection - WorkManager for scheduled background jobs - Kotlin Coroutines + Flow

- OkHttp 4.x with certificate pinning support (mTLS)
- kotlinx.serialization for JSON
- Timber for logging

Output: build.gradle.kts, settings.gradle.kts, AndroidManifest.xml, base Application class with Hilt, and a README with module structure.
Use clean architecture: data / domain / presentation layers.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 1.2 — DeviceScanner Module

□ COPY & PASTE TO CLAUDE

Using the aegis-agent scaffold, implement a DeviceScanner Kotlin class inside agent/data/scanner/. It must:

1. Detect root using 3 methods:
 - a. Check for su binary in common paths
 - b. Check Build.TAGS for "test-keys"
 - c. Check if /system/app/Superuser.apk exists
2. Query the Play Integrity API and return:


```
MEETS_STRONG_INTEGRITY / MEETS_DEVICE_INTEGRITY /
MEETS_BASIC_INTEGRITY / FAILS
```
3. Read android.os.Build.VERSION.SECURITY_PATCH
4. Read bootloader state from SystemProperties

Return a DeviceReport data class serialized as JSON.
All functions must be suspending (coroutines, NOT callbacks).
Add unit tests using JUnit5 + Mockk.

□ Security Note

Play Integrity API is the most reliable signal — treat it as primary. Binary checks (su path) are supplementary. Rooted devices with Magisk Hide will defeat binary checks.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture ·

Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points
· How to Extend. Now wait for me to provide the codebase.

Prompt 1.3 — App Intelligence Collector

□ COPY & PASTE TO CLAUDE

Implement `AppIntelligenceCollector.kt` in `agent/data/apps/`. It must:

1. List all installed packages via `PackageManager`
 - Filter out system packages ONLY for BYOD mode (add a mode flag)
2. For each package collect:
 - `packageName`, `versionName`, `versionCode`
 - SHA-256 of the base APK file
 - Signing certificate SHA-256
 - List of requested permissions
 - `installSource` (Play Store / sideloaded / MDM)
3. Register a `BroadcastReceiver` for `ACTION_PACKAGE_ADDED` and `ACTION_PACKAGE_REMOVED` — emit changes as a `Flow<PackageEvent>`
4. Store last-known app list in `DataStore<Preferences>` for delta diffing.
Only report CHANGED apps on subsequent scans (bandwidth optimisation).

Return `AppSnapshot` data class serialized as JSON. Add KDoc on all public APIs.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 1.4 — Log Filter Agent

This is a new module specific to AEGIS v2. The device never sends all logs to the server — only important ones.

□ COPY & PASTE TO CLAUDE

Implement a `LogFilterAgent.kt` in `agent/data/logs/`.

Requirements:

1. Monitor Android logcat output in real-time using a coroutine + Flow pipeline
2. Define an `ImportanceFilter` that passes only logs matching these criteria:
 - Log tag contains: SECURITY, AUTH, CRASH, ANOMALY, VIOLATION, PERMISSION
 - Log level is ERROR or ASSERT (`Log.e`, `Log.wtf`)
 - Message matches a threat regex list:

```

    * "permission denied", "root", "injection", "exploit",
    * "brute force", "failed login", "certificate error"
3. Discard all other logs immediately – never store or buffer them
4. Buffer passing logs in a local in-memory queue (max 200 entries, 10-minute TTL)
5. Serialize filtered logs as ImportantLog data class:
   timestamp, deviceId, tag, level, message, matchedRule
6. Expose a Flow<List<ImportantLog>> that flushes the buffer every 30 seconds
   (or immediately if ≥ 50 entries accumulate)

```

Add unit tests with mock logcat input.

❏ Security Note

Never send raw logcat to the server. Only pre-filtered important logs leave the device. This protects user privacy and keeps bandwidth low.

❏ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

PH 02

Server Ingestion — All JSON Arrives Here First

❏ Weeks 5–7

The server is the single entry point for all device data. Every piece of information from the agent arrives as JSON here before being routed to analysis, storage, or Kafka topics.

Prompt 2.1 — FastAPI Ingestion Gateway

❏ COPY & PASTE TO CLAUDE

Build a production-grade FastAPI ingestion service in Python 3.11.

Endpoints (all receive JSON):

- POST /api/v1/telemetry	- DeviceReport JSON (device posture)
- POST /api/v1/apps	- AppSnapshot JSON (app intelligence)
- POST /api/v1/logs	- ImportantLogs JSON array (filtered device logs)
- POST /api/v1/apk/upload	- APK binary (multipart), validate magic bytes
- GET /health	
- GET /metrics	- Prometheus

Security:

- mTLS: verify device identity via client certificate, extract device_id from CN field
- Rate limit: 10 requests/minute per device_id (use slowapi)

- Validate all JSON with Pydantic v2 schemas
- Never log raw payload contents (contains device fingerprints)

On receipt:

- Telemetry + Apps → save to PostgreSQL + publish to Kafka "device.telemetry"
- APK → save to MinIO/S3, publish to Kafka "apk.analysis"
- Logs → publish directly to Kafka "logs.stream"
- If device risk flag exists in DB → also publish to "device.flagged"

Include docker-compose.yml: FastAPI + Kafka + Zookeeper + MinIO + PostgreSQL + Redis

Include .env.example and pytest tests for all endpoints.

❏ Security Note

The server is the single source of truth. All device data must pass through here. Kafka routing happens server-side — the device agent only needs one HTTPS endpoint per data type.

❏ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 2.2 — Kafka Topics & Schema Registry

❏ COPY & PASTE TO CLAUDE

Design and implement the Kafka topic architecture for AEGIS v2.

Topics:

- device.telemetry (partitions: 12, retention: 7 days)
- apk.analysis (partitions: 6, retention: 30 days)
- logs.stream (partitions: 12, retention: 24 hours — important logs only)
- risk.scores (partitions: 6, retention: 90 days)
- response.actions (partitions: 3, retention: 7 days)
- device.flagged (partitions: 6, retention: 30 days)

Using the Confluent Kafka Python client and Avro schemas:

1. Write Avro schemas for: DeviceTelemetry, AppSnapshot, ImportantLog, RiskScore
2. Create topic_admin.py script to create/update all topics
3. Create a base KafkaConsumer class with:
 - At-least-once delivery
 - Dead letter queue on 3 failures
 - Graceful shutdown on SIGTERM

❏ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

PH 03

Analysis — Static Engine + AI CLI Agent (Parallel)

□ Weeks 8–12

Both tracks run simultaneously on the same APK. The Python static engine is fast and deterministic (rules, regex, entropy). The AI CLI agent catches subtle logic threats that rules miss. Their outputs are merged into one StaticReport.

Prompt 3.1 — Python Static Analysis Engine

□ COPY & PASTE TO CLAUDE

Build a Python static analysis engine for Android APKs.
It consumes the "apk.analysis" Kafka topic.

Architecture:

1. APKUnpacker: download APK from MinIO, unpack with apktool, decompile with jadx into a temp directory
2. SecretDetector: scan all smali + decompiled Java/Kotlin source for:
 - API keys (regex patterns for AWS, Google, Stripe, etc.)
 - High-entropy strings (Shannon entropy > 4.5 on strings > 20 chars)
 - Hardcoded IPs and URLs
3. PermissionAnalyzer: parse AndroidManifest.xml, score each permission using a risk matrix (DANGEROUS > SIGNATURE > NORMAL)
4. TrackerDetector: match package names against Exodus Privacy tracker DB
5. ObfuscationDetector: measure average identifier length in class names (< 3 chars average = likely obfuscated)
6. StaticReport dataclass: aggregate all findings with severity scores

Wrap in a Celery task. Input: s3_key (APK path).

Output: publish StaticReport JSON to Kafka "risk.scores".

Add Redis as Celery broker.

On completion, set a "static_done" flag in Redis with the APK sha256 as key.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 3.2 — AI CLI Agent (aegis scan)

This runs in parallel with Prompt 3.1 on the same APK. The CLI agent uses Claude to reason about decompiled code and spot threats that pure static rules cannot detect.

□ COPY & PASTE TO CLAUDE

Build a CLI tool named "aegis" using Python + Click + Rich + Anthropic SDK.

Commands:

```
aegis scan <apk_path>
  - Download APK from MinIO using s3_key
  - Decompile with apktool + jadx (reuse output dir if static engine is running)
  - Send decompiled code to Claude API for AI security analysis:
    * Detect business-logic abuse
    * Detect hidden data collection patterns
    * Detect command-and-control callback patterns
    * Detect anti-analysis / anti-debug techniques
  - Display findings in a Rich table with color-coded severity
  - Produce risk score 0-100
  - Set "ai_done" flag in Redis with APK sha256 as key
```

```
aegis report --input <scan_json> --format [pdf|json|html]
  - Generate professional report from scan output
```

```
aegis device <device_id> --api-key <key>
  - Query AEGIS backend for device risk posture
```

```
aegis config set --api-url <url> --api-key <key>
  - Store config in ~/.aegis/config.yaml
```

Merger process (separate worker):

```
- Poll Redis for both "static_done" AND "ai_done" flags for the same sha256
- When both are set, merge StaticReport + AI findings into a FinalReport
- Publish FinalReport to Kafka "risk.scores"
```

Use claude-sonnet-4-20250514. Temperature 0.2 (factual output).

Include pyproject.toml. Add README with install and usage instructions.

□ Security Note

The AI agent must be given only the decompiled code — never the raw APK binary. Keep prompts focused: send one class/module at a time if the codebase is large, not the entire decompiled output in one call.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

PH 04

Log Streaming — AI Agent on Important Logs Only

□ Weeks 13–15

Important logs are already filtered on-device (Phase 1). Here they are streamed from the server via Kafka and analyzed by an AI agent. No routine logs are ever stored or processed.

Prompt 4.1 — Log Stream Processor

□ COPY & PASTE TO CLAUDE

Build a Python log stream processor that consumes the Kafka "logs.stream" topic.

LogStreamProcessor:

1. Consume ImportantLog messages from logs.stream in real-time
2. Apply a second-pass filter (logs already filtered on-device, but verify):
 - Confirm message matches threat keyword list
 - Skip any log that is a duplicate of a log seen for the same device in the last 60 seconds (use Redis for dedup with 60s TTL key: device_id:log_hash)
3. Group logs by device_id in a 5-second tumbling window
4. After each window, if any logs remain → forward the batch to the AI Log Analyst
5. Track stats: total received, passed filter, sent to AI (expose on /metrics)

The processor must be stateless and horizontally scalable (Kafka consumer groups). Add Docker deployment configuration.

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 4.2 — AI Log Analyst Agent

□ COPY & PASTE TO CLAUDE

Build a Python AI Log Analyst that receives batches of important logs per device.

AILogAnalyst:


```

1. Receive a batch: {device_id, logs: [ImportantLog], device_risk_context: RiskScore}
2. Build a structured prompt for Claude API:
  - Include current device risk score as context
  - Include the log batch
  - Ask Claude to detect:
    * Privilege escalation attempts
    * Repeated authentication failures (brute force patterns)
    * Unusual process spawning
    * Data exfiltration indicators (large outbound bursts)
    * Permission abuse patterns
    * Crash loops that may indicate exploit attempts
3. Parse Claude response into a LogThreatReport:
  threat_type, severity (LOW/MED/HIGH/CRITICAL), confidence, description, raw_evidence
4. Publish LogThreatReport to Kafka "risk.scores"

```

```

Use claude-sonnet-4-20250514. Temperature 0.2.
Rate limit: 100 Claude calls / hour (Redis counter with 1h TTL).
Store all prompts + responses in PostgreSQL for audit trail.
Add pytest tests with mocked Claude responses.

```

❏ Security Note

The AI agent only sees pre-filtered important logs — never raw device logcat. This protects privacy and keeps AI costs low. The dedup step in the stream processor prevents the same event from triggering multiple AI calls.

❏ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

PH 05

Risk Engine, SOC Dashboard & Response Automation

❏ Weeks 16–20

Aggregate all signals into composite risk scores, expose them through a real-time SOC dashboard, and automatically enforce MDM responses when thresholds are crossed.

Prompt 5.1 — Risk Scoring Engine

❏ COPY & PASTE TO CLAUDE

Build the AEGIS Risk Engine in Python. It consumes the Kafka "risk.scores" topic and computes a composite RiskScore per device and per app.

```

Formula:
    RiskScore = (0.35 * StaticAnalysis) + (0.35 * DevicePosture)
               + (0.20 * LogThreats) + (0.10 * ThreatIntel)

DevicePosture sub-scoring:
    +40 pts  if is_rooted == True
    +30 pts  if play_integrity == FAILS
    +20 pts  if security_patch is older than 90 days
    +10 pts  if bootloader == unlocked

StaticAnalysis sub-scoring (from FinalReport):
    +25 pts  if secrets found
    +20 pts  if obfuscation detected
    +per-permission risk from permission_risk_matrix.json
    +15 pts  if known tracker SDKs detected
    + AI findings severity score (from CLI agent)

LogThreats: AILogAnalyst severity → mapped to 0-40 pts

ThreatIntel: async VirusTotal hash lookup
    +50 pts  if VT detections > 3

Output: RiskScore(device_id, app_id, score, severity, breakdown, timestamp)
Persist to PostgreSQL.
Publish to Kafka "response.actions" if score >= 75.
Alert deduplication: same device+app pair silent for 4 hours unless score
increases by >= 10 points.

```

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 5.2 — SOC Dashboard (API + Frontend)

□ COPY & PASTE TO CLAUDE

Build the AEGIS SOC Dashboard.

```

FastAPI Backend:
    GET  /api/v1/devices           - paginated device fleet list with risk scores
    GET  /api/v1/devices/{id}     - device detail: posture, apps, alert history,
logs
    GET  /api/v1/devices/{id}/apps - apps installed on device with risk scores
    GET  /api/v1/alerts           - alert feed (filterable by severity/date)
    GET  /api/v1/alerts/{id}      - alert detail with full risk breakdown
    POST /api/v1/response/{alert_id} - trigger response action
    WS   /ws/alerts               - WebSocket real-time alert stream

```

```
Auth: JWT Bearer tokens (HS256). GET /auth/login endpoint.
Roles: ANALYST (read only), RESPONDER (can trigger actions), ADMIN.
PostgreSQL with SQLAlchemy 2.0 async. Alembic for migrations.

React Frontend (Vite + TypeScript + TailwindCSS dark theme):
  /dashboard    - KPI cards (safe/at-risk/critical) + risk histogram + live ticker
  /devices      - Sortable/filterable fleet table with risk badges
  /devices/:id  - Posture card + installed apps + alert timeline + log viewer
                  + Action buttons: Quarantine / Flag for Review
  /alerts       - Severity filter tabs: All / Critical / High / Medium

Color system: red=critical, orange=high, yellow=medium, green=safe.
```

□ UNDERSTANDING CHECKPOINT

Paste this block into Claude after receiving the generated code:

You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Prompt 5.3 — MDM Response Engine

□ COPY & PASTE TO CLAUDE

Build the AEGIS Response Engine. It consumes the Kafka "response.actions" topic.

```
Action types (enum):
  ALERT_SOC          - send alert to SOC dashboard WebSocket (always executed)
  RESTRICT_NETWORK   - push Intune compliance policy to restrict network access
  QUARANTINE          - mark device non-compliant in Intune via Graph API
  CREATE_TICKET       - auto-create Jira incident ticket with risk breakdown
  NOTIFY_USER        - push FCM notification to device
```

Severity routing:

```
MEDIUM  → ALERT_SOC + CREATE_TICKET
HIGH     → above + NOTIFY_USER + RESTRICT_NETWORK
CRITICAL → all above + QUARANTINE
```

Intune Integration (Microsoft Graph API):

```
- Auth via Azure AD app registration (client_credentials flow)
- Mark device non-compliant: PATCH /deviceManagement/managedDevices/{id}
```

Jira Integration:

```
- Create issue in project AEGIS with severity-mapped priority
- Include full risk score breakdown in description
```

```
Log all actions to PostgreSQL with status (success/failed).
Allow manual retry from SOC dashboard.
```

```
Add dry_run mode: log what would happen without calling external APIs.
```

□ **UNDERSTANDING CHECKPOINT**

Paste this block into Claude after receiving the generated code:
 You are a Staff-Level Software Engineer, Security Engineer, and Technical Educator. Reverse engineer and document the code I will provide. Make me FULLY UNDERSTAND the system as if I built it myself. Produce: Overview · Architecture · Execution Flow · File Breakdown · Key Components · Security Analysis · Weak Points · How to Extend. Now wait for me to provide the codebase.

Appendix — Quick Reference

Technology Stack Summary

Layer	Technology	Why
Android Agent	Kotlin + Hilt + WorkManager	Native Android; reliable background jobs; DI
Log Filtering	Kotlin + Logcat + Coroutines	On-device filter — only important logs leave device
Transport	JSON over HTTPS mTLS	All data from agent → server as structured JSON
API Gateway	FastAPI + Python 3.11	Fast async; Pydantic v2 validation; OpenAPI docs
Message Bus	Apache Kafka	High-throughput, durable; decouples microservices
Static Engine	apktool + jadx + Celery	Industry standard APK tools; async job processing
AI Analysis	Claude API (claude-sonnet-4)	Logic threat detection + log anomaly analysis
AI CLI	Python + Click + Rich	Human-readable scan output; parallel with static engine
Risk Engine	Python + PostgreSQL	Composite scoring; VirusTotal integration
Frontend	React + TypeScript + Vite	Fast builds; type safety; WebSocket live alerts
Response	Intune + Graph API + Jira	Automated MDM enforcement on high-risk devices
Storage	PostgreSQL + MinIO + Redis	Relational for metadata; object for APKs; cache/queue

Prompt Sequence Index

#	Phase	Prompt	Output
1.1	PH1 — Agent	Project Scaffold & Gradle Setup	build files, Hilt DI
1.2	PH1 — Agent	DeviceScanner Module	root, Play Integrity, DeviceReport JSON

1.3	PH1 — Agent	AppIntelligenceCollector	AppSnapshot JSON, delta diffing
1.4	PH1 — Agent	Log Filter Agent	ImportantLogs JSON, on-device filter
2.1	PH2 — Ingestion	FastAPI Ingestion Gateway	REST API + Kafka routing
2.2	PH2 — Ingestion	Kafka Topics & Schema Registry	Avro schemas, 6 topics
3.1	PH3 — Analysis	Python Static Analyzer (parallel A)	StaticReport, Celery tasks
3.2	PH3 — Analysis	AI CLI Agent aegis scan (parallel B)	AI findings, merged FinalReport
4.1	PH4 — Logs	Log Stream Processor	Kafka consumer, dedup, windowing
4.2	PH4 — Logs	AI Log Analyst Agent	LogThreatReport, audit trail
5.1	PH5 — Risk	Risk Scoring Engine	RiskScore, VirusTotal, dedup
5.2	PH5 — SOC	SOC Dashboard (API + Frontend)	FastAPI REST+WS, React UI
5.3	PH5 — Response	MDM Response Engine	Intune quarantine, Jira tickets

Architecture Changes from v1

- Dynamic analysis (Frida / Docker emulator) removed — can be added as Phase 6 if needed later
- All device data now travels as JSON → server first, before any Kafka routing or analysis
- Static Python engine and AI CLI agent run in parallel (same phase, same APK trigger)
- Log analysis uses streaming: only important logs (tagged SECURITY/CRASH/ANOMALY) are processed
- Two-stage log filtering: on-device (Kotlin agent) + dedup pass (stream processor) — AI sees only what matters
- Risk formula updated: LogThreats replaces BehaviorAnomaly from v1 (simpler, no sandbox needed)

AEGIS v2 — 5 Phases · 13 Prompts · ~20 Weeks · Simplified Architecture